# CAutomation Project Client Contract

### Authoritative platform-level business contract

Status: Final authoritative project-level platform contract for CAutomation. This document is intentionally platform-level. Module-specific requirements, screens, entities, endpoints, data fields, schemas, workflows, and acceptance details belong in module-level contracts. When a project-level rule and a module-level detail conflict, the project-level rule controls unless this project-level contract is formally changed.

Authoring rule: every section is written to be decision-complete for platform-level architecture. If required information is missing at module level, the AI must report the gap instead of inventing module behavior.

## 1. Document Purpose and Decision Boundary

This contract defines the business and product authority for the complete CAutomation platform. It prevents the AI from treating module contracts as isolated applications and prevents project-level gaps from being filled with invented module details.

- Use this document to decide platform scope, shared concepts, ownership, lifecycle, governance, cross-module rules, business quality requirements, constraints, and platform acceptance.
- Do not use this document to infer module-specific fields, screens, reports, endpoints, database tables, detailed domain workflows, or acceptance criteria.
- If this contract defines a shared rule, every module must obey it. If this contract is silent about a module detail, the AI must report a module-contract gap instead of inventing the answer.

| Topic | Project-level decision | Not decided here |
|---|---|---|
| Business scope | The platform manages clients, projects, modules, contracts, AI executions, generated deliverables, validation evidence, review, approval, export, and change history. | Exact domain behavior inside Pipeline Management or User & Client Management. |
| Authority | Approved project-level contracts define shared behavior for every module. | Module-specific exceptions unless explicitly approved at project level. |
| Output expectation | One end-to-end web application with shared navigation, authorization, traceability, review, approval, reports, and export behavior. | Pixel-perfect UI details or implementation-specific naming beyond CAutomation terminology. |

## 2. Platform Vision

CAutomation is a contract-driven web application platform and AI Engine reference project. It must be useful as an operational application and as the proving ground for controlled AI-assisted generation.

- The product vision is controlled, auditable, repeatable AI generation based on approved contracts, not ad hoc prompting.
- The platform connects human-authored contracts to normalized inputs, pipeline executions, generated outputs, validation results, review decisions, approvals, and exports.
- The platform supports multiple clients, multiple projects per client, and multiple modules per project without mixing ownership, visibility, lifecycle state, or acceptance history.
- Uncertainty must be visible: missing contracts, invalid inputs, failed validations, unresolved warnings, rejected outputs, and unapproved deliverables are first-class business states.
- Human approval remains required for business-significant contract and deliverable decisions; AI output is evidence-backed input to review, not automatic business authority.

## 3. Stakeholders and Responsibilities

The platform must distinguish governance ownership, client administration, project decisions, technical operation, review authority, and AI execution operation.

| Stakeholder | Business responsibility | Decision authority | Cannot decide |
|---|---|---|---|
| Platform Owner | Owns product direction, platform-wide rules, module model, contract model, acceptance model, release readiness, and naming standards. | Platform-level behavior and governance. | Client-specific business content or module-specific details without module contract authority. |
| System Administrator | Maintains operational settings, platform availability, backups, deployment, provider configuration, and technical administration. | Operational administration. | Business approval of contracts or deliverables unless separately assigned. |
| Client Administrator | Manages a client account, client users, client projects, and client visibility boundaries. | Client membership and client project administration. | Other clients or platform governance. |

| Stakeholder | Business responsibility | Decision authority | Cannot decide |
|---|---|---|---|
| Project Owner | Owns project contracts, selected modules, execution readiness, reviews, and final acceptance. | Project lifecycle and deliverable approval. | Security, traceability, or validation rules. |
| Project Member | Contributes to contracts, reviews, and assigned project/module work. | Assigned tasks within project permissions. | Global platform decisions. |
| Reviewer / Approver | Reviews generated deliverables, reports, and validation evidence. | Approval, rejection, or requested rework within assigned scope. | Silent changes to source contracts or overrides of failed validation. |
| AI Engine Operator | Runs permitted pipelines and inspects execution results. | Execution initiation and operational review. | Acceptance of invalid outputs or cross-client access. |

## 4. Platform Concepts

These concepts are shared by every module and must be used consistently across navigation, permissions, lifecycle state, reporting, generated code, and documentation.

| Concept | Platform meaning | Architectural consequence |
|---|---|---|
| Client | A business customer or organizational owner of projects, users, contracts, executions, reports, and deliverables. | Client-owned data must be isolated by membership and permissions. |
| Project | A bounded automation initiative under a client. It contains project contracts, selected modules, module contracts, executions, reports, deliverables, and approvals. | Project context must be present in module workflows and execution history. |
| Module | A functional domain within a project, such as Pipeline Management or User & Client Management. | Modules share platform rules but own domain-specific requirements and technical details. |
| Contract | A human-approved source document used as generation authority. | Contracts require type, scope, owner, version, status, approval evidence, and execution traceability. |
| Pipeline | An ordered process of tasks that transforms approved inputs into outputs, reports, or deliverables. | Execution state, task evidence, and generated artifacts must be inspectable. |
| Execution | One run of a pipeline or task against a defined approved input set. | Every generated artifact must be traceable to an execution identity. |
| Deliverable | An output intended for review, approval, export, or application. | Deliverables cannot become accepted without required validation evidence. |
| Validation evidence | Structured proof that an input, execution, output, or deliverable satisfied required checks. | Failed or missing evidence blocks acceptance unless an explicit approved waiver policy exists. |
| Approval | An attributable business decision that accepts or rejects a contract, execution output, waiver, or deliverable. | Approval must be explicit, timestamped, permission-controlled, and auditable. |

## 5. Platform Modules and Scope Separation

CAutomation currently has two reference modules: Pipeline Management and User & Client Management. The project contract defines how modules coexist; module contracts define what each module does.

- Pipeline Management owns pipeline, task, execution, report, validation, deliverable, and export behavior at module level.
- User & Client Management owns users, clients, roles, memberships, client/project administration, and related administration workflows at module level.
- Shared platform navigation must present modules as parts of one application, not separate systems.
- A module may depend on Client, Project, Role, Permission, Contract, Execution, Report, Deliverable, Approval, and Validation Evidence, but it must not redefine them inconsistently.
- Project-level contracts must not contain detailed module requirements; they define the rules every module must obey.
- A new module may be added only by providing module-level contracts that inherit this project-level model.

## 6. Contract Model and Source-of-Truth Rules

The platform uses four canonical contract types: two at project level and two at module level. The AI must not add additional contract types unless the project-level model is formally changed.

- Only approved contracts are authoritative. Draft, obsolete, rejected, or superseded contracts must not be used as generation authority.
- The platform must preserve contract identity, type, scope, owner, version, status, approval evidence, source file, normalized representation, and relationship to executions.

- A generated result must state exactly which contract versions and normalized inputs were used.
- Changing an approved contract creates a new version or supersession path; it must not silently rewrite prior execution history.

| Contract | Purpose | Decision boundary |
|---|---|---|
| Project Client Contract | Business and product authority for the whole platform. | Platform-wide business rules, concepts, quality attributes, constraints, acceptance, and glossary. |
| Project Engineering Contract | Engineering authority for the whole platform. | Architecture, stack, security, API, data, logging, testing, deployment, AI generation standards. |
| Software Requirements Specification | Module-level business requirements. | Module-specific capabilities, workflows, actors, acceptance criteria, screens, reports, and business details. |
| Architecture and Technical Specification | Module-level technical requirements. | Module-specific APIs, data structures, UI details, integration details, and technical behavior. |

## 7. Cross-Module Business Rules

These rules apply across all modules and prevent inconsistent business assumptions.

- Client isolation: users may only see clients, projects, contracts, executions, reports, and deliverables they are authorized to access.
- Project context: actions that affect project-owned data must occur inside a selected project.
- Approval before authority: contracts and deliverables are not authoritative until approved by an authorized role.
- Validation before acceptance: generated deliverables cannot be accepted when required validation evidence is missing or failed.
- Traceability cannot be optional: every generated artifact, report, approval, export, and waiver must be traceable to source contracts and execution history.
- No hidden state transitions: draft, ready, running, passed, failed, reviewed, approved, rejected, exported, archived, and superseded states must be visible when they affect decisions.
- Modules must not bypass platform roles, permissions, lifecycle status, validation blockers, or audit requirements.
- Deletion must be controlled. Business-significant records should normally be archived or superseded rather than silently removed.

## 8. Platform Workflows

The platform must support end-to-end business workflows that connect clients, projects, contracts, modules, generation, review, approval, and export.

| Workflow | Required platform behavior | Completion state |
|---|---|---|
| Client onboarding | Create or administer a client, assign administrators, establish visibility boundaries. | Client active and manageable. |
| Project setup | Create project under a client, choose modules, attach project contracts, assign responsible roles. | Project ready for contract review. |
| Contract approval | Review contracts, mark approved/rejected/superseded, preserve version history. | Approved contract set available for execution. |
| Execution preparation | Check required contract availability, approval status, scope, and compatibility before a pipeline run. | Execution can start or readiness gaps are shown. |
| AI generation | Run authorized pipeline tasks using approved inputs and produce traceable outputs. | Execution passed, failed, or requires review. |
| Validation and review | Inspect reports, validation evidence, generated artifacts, blockers, and warnings. | Reviewer decision recorded. |
| Deliverable approval/export | Approve acceptable deliverables and export packages with evidence and provenance. | Deliverable package accepted and auditable. |
| Change management | Supersede contracts, rerun affected pipelines, preserve prior evidence. | New baseline replaces old baseline with traceability. |

## 9. Business Quality Requirements

The platform must be judged by business quality, not only by technical correctness.

| Quality attribute | Required business meaning | Acceptance signal |
|---|---|---|
| Traceability | Users can follow every deliverable back to contracts, normalized inputs, pipeline, task, execution, and validation evidence. | No accepted deliverable lacks provenance. |
| Clarity | Users can understand current status, next action, owner, and blocker without reading logs. | Important states are visible in the UI. |

| Quality attribute | Required business meaning | Acceptance signal |
|---|---|---|
| Consistency | Shared concepts behave the same across modules. | Same lifecycle and permission rules apply everywhere. |
| Governance | Approval, rejection, supersession, waiver, and export are controlled business actions. | Decision history is attributable. |
| Recoverability | Failed or invalid executions do not corrupt accepted baselines. | Users can inspect failure and rerun after correction. |
| Usability | The platform supports normal business use by non-developer stakeholders. | Core workflows are understandable without code knowledge. |
| Extensibility | New modules can be added without redefining platform concepts. | Module additions preserve shared rules. |

## 10. Constraints and Assumptions

These constraints keep the AI from inventing conflicting platform requirements.

- CAutomation is the only project name. Legacy names such as CFF, CFFP, CRUD Factory, or AI Factory must not appear in generated platform behavior.
- PDF is the primary supported contract source format at this stage. DOCX may remain supported where already implemented, but project decisions must not assume equal engineering priority.
- The platform must be web-based and end-to-end: frontend, backend API, persistence, authentication, authorization, reports, validation, and export behavior must work together.
- Human approval is required for business-significant contract, waiver, baseline, export, and deliverable decisions.
- The platform must prefer explicit failure over silent fallback when required information is missing.
- Project-level contracts stay platform-level; module-specific requirements stay in module contracts.
- The current platform does not require production infrastructure choices beyond safe configuration, auditability, and deployability.

## 11. Acceptance Criteria

The complete CAutomation platform is acceptable only when platform-wide business behavior is generated coherently and can be validated end-to-end.

- The application supports client, project, module, contract, execution, report, deliverable, review, approval, waiver, and export concepts as one coherent platform.
- Project-level and module-level contract responsibilities remain clearly separated in generated behavior.
- Users cannot access data outside their authorization boundary.
- Pipeline outputs and deliverables are traceable to approved source contracts, normalized inputs, execution identity, and validation evidence.
- Invalid, insufficient, or unapproved inputs are surfaced before downstream generation or acceptance.
- Failed validations block acceptance until corrected, rerun successfully, or waived by an explicit approved policy.
- The platform UI exposes status, ownership, blockers, provenance, and next actions for key workflows.
- Generated documentation, reports, and exports use current CAutomation naming and the approved contract model only.

## 12. Project-Level Glossary

The AI must use these terms consistently in generated platform behavior and must not substitute legacy terminology.

| Term | Platform definition |
|---|---|
| CAutomation | The complete contract-driven AI Engine platform and reference web application. |
| Project-level contract | A contract that defines behavior shared across all modules in a project. |
| Module-level contract | A contract that defines requirements or technical details for one module only. |
| Decision-complete | Sufficiently explicit that the AI can make the correct platform-level decision without guessing. |
| Platform-level | Shared across the entire application and every module. |
| Module-specific | Owned by a single module and excluded from project-level contracts. |
| Normalized input | Structured representation of approved source documents used by downstream tasks. |
| Validation evidence | Structured proof that a contract, input, execution, output, deliverable, or waiver satisfied required checks. |
| Deliverable package | Generated outputs plus reports, manifests, evidence, and provenance intended for review, approval, or export. |

| Term | Platform definition |
| --- | --- |
| Superseded | No longer current but preserved for traceability and history. |
| Waiver | Explicit authorized decision to accept a known validation exception under a controlled policy. |